

ASYNCHRONOUS COUNTER

Inputs			
Signal	Width	MSB	LSB
clk	1 bit	clk	clk
reset	1 bit	reset	reset
Outputs			
Signal	Width	MSB	LSB
q	4-bit bus	q[3]	q[0]

MAIN CODE

```
module async_counter (  
    input wire    clk,  
    input wire    reset,  
    output reg [3:0] q  
);  
  
    // Asynchronous 4-bit counter using JK flip-flop concept  
  
    always @(posedge clk or posedge reset)  
    begin  
        if (reset)  
            q[0] <= 1'b0;  
        else  
            q[0] <= ~q[0];  
        end  
  
    always @(negedge q[0] or posedge reset)  
    begin  
        if (reset)
```

```

        q[1] <= 1'b0;
    else
        q[1] <= ~q[1];
    end

    always @(negedge q[1] or posedge reset)
    begin
        if (reset)
            q[2] <= 1'b0;
        else
            q[2] <= ~q[2];
        end

        always @(negedge q[2] or posedge reset)
        begin
            if (reset)
                q[3] <= 1'b0;
            else
                q[3] <= ~q[3];
            end

        endmodule

```

Test Bench CODE

```
`timescale 1ns/1ps
```

```
module async_counter_tb;
```

```
    reg clk;
```

```
    reg reset;
```

```
wire [3:0] q;

// Instantiate counter
async_counter uut (
    .clk(clk),
    .reset(reset),
    .q(q)
);

// Clock generation
always #10 clk = ~clk;

initial
begin
    // Initial values
    clk = 0;
    reset = 0;

    // Apply reset pulse
    #10;
    reset = 1;

    #20;
    reset = 0;

    // Run counter
    #500;

    $finish;
end
```

```
// Display output  
  
initial  
  
begin  
    $monitor("Time = %0t | CLK = %b | RESET = %b | Q = %b",  
        $time, clk, reset, q);  
end  
  
endmodule
```